

Kelvin Aparecido Lima Albuquerque

**VERIFICAÇÃO DE CENAS USANDO UM
ALGORITMO DE VISÃO COMPUTACIONAL**

Campinas

2026

Kelvin Aparecido Lima Albuquerque

VERIFICAÇÃO DE CENAS USANDO UM ALGORITMO DE VISÃO COMPUTACIONAL

Trabalho de Conclusão de Curso apresentado
como parte dos requisitos para obtenção do di-
ploma do Curso Tecnólogo em Análise e Desen-
volvimento de Sistemas do Instituto Federal de
Educação, Ciência e Tecnologia de São Paulo -
Campus Campinas.

Instituto Federal de Educação, Ciência e Tecnologia de São Paulo

Campus Campinas

Curso Tecnólogo em Análise e Desenvolvimento de Sistemas

Orientador: Prof. Dr. Ricardo Sovat

Campinas

2026

Albuquerque, Kelvin Aparecido Lima de
VERIFICAÇÃO DE CENAS USANDO UM ALGORITMO DE VISÃO COMPUTACIONAL / Kelvin Aparecido Lima Albuquerque. – Campinas, 2026.
43 f. : il. (algumas color.) ; 30 cm.
Orientador: Prof. Dr. Ricardo Sovat
Trabalho de Conclusão de Curso (Tecnólogo em Análise e Desenvolvimento de Sistemas) – Instituto Federal de Educação, Ciência e Tecnologia de São Paulo - Campus Campinas, 2026.
1. Visão Computacional. 2. Detecção de Objetos. 3. YOLO. 4. Inteligência Artificial. I. Sovat, Ricardo, orient. II. Instituto Federal de Educação, Ciência e Tecnologia de São Paulo - Campus Campinas. III. Título.

Kelvin Aparecido Lima Albuquerque

VERIFICAÇÃO DE CENAS USANDO UM ALGORITMO DE VISÃO COMPUTACIONAL

Trabalho de Conclusão de Curso apresentado
como parte dos requisitos para obtenção do di-
ploma do Curso Tecnólogo em Análise e Desen-
volvimento de Sistemas do Instituto Federal de
Educação, Ciência e Tecnologia de São Paulo -
Campus Campinas.

Aprovado pela banca examinadora em: ____ de ____ de ____.

BANCA EXAMINADORA

Prof. Dr. Ricardo Sovat
Orientador - IFSP Campus Campinas

Profa. Ma. xxxxxxxxxxxxxxxxx
Universidade Estadual de Campinas

Prof. Me. xxxxxxxxxxxxxxxxx
IFSP Campus XXXX

*Dedico este trabalho aos meus familiares,
colegas de classe, professores e servidores do Instituto
que colaboraram em minha jornada formativa.*

Agradecimentos

Agradeço primeiramente a Deus, pelo dom da vida e pela oportunidade de concluir mais uma etapa de minha experiência acadêmica.

Agradeço também à minha família, que deu todo o apoio necessário para que eu chegasse até aqui.

Agradeço ao meu orientador, Prof. Dr. Ricardo Sovat, que me auxiliou a solucionar as dificuldades encontradas no caminho.

Agradeço a todos os professores e servidores do IFSP Campus Campinas, que contribuíram direta e indiretamente para a conclusão deste trabalho.

*“O que prevemos raramente ocorre;
o que menos esperamos
geralmente acontece.”
– Benjamin Disraeli*

RESUMO

Este trabalho apresenta o desenvolvimento de um sistema baseado em visão computacional para análise automatizada de ambientes por meio da detecção de objetos em imagens e vídeos. A proposta integra uma aplicação mobile, um backend baseado em arquitetura de serviços e um modelo de inteligência artificial treinado com a arquitetura YOLO (*You Only Look Once*). O sistema permite a captura de imagens por dispositivos móveis, envio para processamento e retorno com a identificação dos objetos detectados. Como diferencial, propõe-se a análise de inconsistências contextuais, possibilitando identificar a ausência de elementos esperados em determinados cenários, como a presença de sinalização sem o respectivo equipamento associado. A metodologia envolve o treinamento de modelos com datasets personalizados, desenvolvimento de uma API RESTful para processamento das requisições e integração com aplicação Android. A avaliação foi realizada por meio de métricas como precisão, recall e F1-score, além de testes práticos em cenários reais de engenharia civil. Como resultado obtido, obteve-se uma solução eficiente, aplicável em processos de inspeção automatizada de canteiros de obras, contribuindo para a prevenção de acidentes e aumento da confiabilidade das análises.

Palavras-chave: visão computacional. detecção de objetos. YOLO. inteligência artificial. inspeção automatizada. segurança do trabalho.

ABSTRACT

This work presents the development of a computer vision-based system for automated environment analysis through object detection in images and videos. The proposal integrates a mobile application, a backend based on service architecture, and an artificial intelligence model trained using the YOLO (*You Only Look Once*) architecture. The system allows images to be captured by mobile devices, sent for processing, and returned with the identified detected objects. As a distinguishing feature, the work proposes contextual inconsistency analysis, making it possible to identify the absence of expected elements in specific scenarios, such as the presence of signage without its corresponding associated equipment. The methodology involves training models with custom datasets, developing a RESTful API to process requests, and integrating it with an Android application. The evaluation was carried out using metrics such as precision, recall, and F1-score, in addition to practical tests in civil engineering real-world scenarios. As a result, the work obtained an efficient, applicable solution for automated inspection processes on construction sites, contributing to accident prevention and analysis reliability.

Keywords: computer vision. object detection. YOLO. artificial intelligence. automated inspection. occupational safety.

LISTA DE ILUSTRAÇÕES

Figura 1 – Log de inicialização do servidor backend com OpenVINO	25
Figura 2 – Tela de autenticação da aplicação móvel	25
Figura 3 – Tela de menu principal de opções	26
Figura 4 – Tela de interface inicial de captura	27
Figura 5 – Tela de confirmação e envio da mídia	28
Figura 6 – Tela de exibição dos laudos contextuais	29
Figura 7 – Tela de reenvio de imagens para inferência	30
Figura 8 – Caixa delimitadora na detecção de cadeira individual	32
Figura 9 – Detecção simultânea de múltiplas cadeiras em sala de reuniões	33
Figura 10 – Detecção correta de extintor de incêndio sob sinalização adequada	34
Figura 11 – Alerta visual do sistema para extintor ausente no suporte	35
Figura 12 – Curva de Precisão-Revocação do modelo de prevenção de incêndio	37

LISTA DE TABELAS

Tabela 1 – Métricas estatísticas de desempenho do modelo YOLO	36
Tabela 2 – Cronograma de atividades	42

SUMÁRIO

1	INTRODUÇÃO	13
2	FUNDAMENTAÇÃO TEÓRICA	15
2.1	Inteligência Artificial e Aprendizado de Máquina	15
2.2	Visão Computacional e Detecção de Objetos	15
2.3	Evolução e Arquitetura YOLO (<i>You Only Look Once</i>)	15
2.4	Processamento de Vídeo, Stride e Otimização com OpenVINO	16
2.5	Arquitetura de APIs RESTful e Framework FastAPI	17
2.6	Mecanismos de Segurança em APIs Web	17
2.7	Modelos de Linguagem Local (LLM) na Usabilidade de Sistemas	18
2.8	Conformidade em Segurança na Engenharia Civil	18
2.9	O Impacto da Segurança no Bem-estar do Trabalhador	19
3	METODOLOGIA	20
3.1	Levantamento Bibliográfico	20
3.2	Preparação e Organização dos Dados (Dataset)	20
3.3	Treinamento do Modelo YOLO	21
3.4	Desenvolvimento da API Backend	21
3.5	Desenvolvimento do Aplicativo Android	22
3.6	Integração entre os Componentes e Segurança	22
3.7	Execução dos Testes Controlados	22
3.8	Avaliação Lógica e Métricas Estatísticas do Sistema	23
4	RESULTADOS	24
4.1	Execução Sistêmica da Arquitetura	24
4.2	Programa Piloto de Detecção (Classe: Cadeira)	31
4.3	Inspeção de Conformidade em Canteiro de Obras (Classe: Extintor)	31
4.3.1	Cenário A: Conformidade em Prevenção de Incêndio	31
4.3.2	Cenário B: Não Conformidade de Segurança (Extintor Ausente)	31
4.4	Consolidação das Métricas Estatísticas	36
4.5	Análise Crítica dos Resultados	36
5	CONSIDERAÇÕES FINAIS	39
5.1	Limitações do Sistema	39
5.2	Sugestões de Trabalhos Futuros	40
	REFERÊNCIAS	41

Cronograma de Atividades	42
A – LISTA DE REFERÊNCIAS COMPLEMENTARES E DOCUMENTAÇÕES DE APIS	43

1 INTRODUÇÃO

O avanço da Inteligência Artificial (IA) e, em particular, das técnicas de visão computacional tem possibilitado o desenvolvimento de soluções automáticas capazes de realizar tarefas complexas que antes dependiam exclusivamente da percepção e do julgamento humano. Entre essas técnicas, destacam-se os modelos de detecção de objetos em tempo real, cujo marco evolutivo é a família de algoritmos YOLO (*You Only Look Once*), amplamente adotada em aplicações industriais, de vigilância e de monitoramento de ambientes (REDMON et al., 2016; TERVEN; CÓRDOVA-ESPARZA, 2023).

A verificação e a auditoria de conformidade física em ambientes de trabalho ou produção constituem etapas críticas para garantir que normas regulamentadoras, padrões de qualidade ou requisitos de segurança estejam sendo estritamente seguidos. Tradicionalmente, o processo de “inspeção de conformidade” consiste na verificação visual sistemática de um cenário por engenheiros, técnicos ou inspetores treinados, buscando atestar a presença de elementos regulamentares (como equipamentos de segurança, sinalização ou maquinário correto) e a ausência de irregularidades ou riscos estruturais.

No entanto, a inspeção exclusivamente manual e visual está intrinsecamente sujeita a falhas cognitivas e humanas. Fatores como a fadiga decorrente de longas jornadas de trabalho, a sobrecarga de atenção em ambientes dinâmicos de grande extensão e a subjetividade inerente à interpretação visual humana criam margens significativas para erros (RAHMAN; ULUSOY; MEYDANLI, 2021). Em canteiros de obras de engenharia civil, por exemplo, a vastidão física das frentes de trabalho impede que os supervisores de segurança monitorem de forma contínua e em tempo real todos os colaboradores simultaneamente. Consequentemente, não conformidades como o não uso de Equipamentos de Proteção Individual (EPIs) ou a ausência de extintores de incêndio e sinalizações obrigatórias podem passar despercebidas, gerando graves riscos à integridade física do trabalhador e passivos jurídicos expressivos para as corporações.

Nesse panorama, define-se o seguinte problema de pesquisa: **Como automatizar a verificação de conformidade em ambientes por meio da detecção inteligente de objetos e análise lógica contextual a partir de imagens digitais?**

A hipótese que norteia este trabalho é que a integração de modelos de visão computacional (YOLO) com uma infraestrutura de comunicação baseada em APIs RESTful e aplicações móveis de captura permite não apenas a localização e classificação precisa de objetos em tempo real, mas também a avaliação lógica automatizada de regras de conformidade do cenário, reduzindo substancialmente a latência de inspeção e as taxas de falha humana.

Cabe salientar que, embora a segurança do trabalho em canteiros de obras de engenharia civil (como a detecção de EPIs ou de elementos de sinalização de segurança) seja adotada como o

principal cenário experimental e de validação deste projeto, a solução desenvolvida foi projetada sob uma arquitetura de microsserviços modular e genérica. Isso significa que o modelo e o motor lógico de análise contextual podem ser reconfigurados e aplicados a diversos outros contextos onde a verificação de conformidade visual se faça necessária, tais como auditorias de qualidade em linhas de montagem industriais, contagem de estoque comercial, verificação de arranjo físico em salas hospitalares ou auditoria de equipamentos em TI.

O objetivo geral deste trabalho é desenvolver, implementar e testar um sistema completo de visão computacional voltado para a inspeção de conformidade de cenas. O sistema é composto por um aplicativo cliente Android escrito em Kotlin, um serviço backend FastAPI de processamento distribuído com suporte a autenticação por token JWT de duas camadas, e uma arquitetura YOLO integrada a um analisador contextual que identifica ausências ou inconsistências lógicas no cenário.

Para estruturar a validação científica deste trabalho, os objetivos específicos contemplam:

- Realizar um levantamento bibliográfico acerca do estado da arte de algoritmos YOLO e sistemas de monitoramento automatizados;
- Elaborar e preparar conjuntos de dados personalizados para o treinamento de um modelo YOLO com foco em elementos de segurança do trabalho;
- Desenvolver a API RESTful backend responsável por receber dados multimídia, autenticar requisições de forma robusta e gerir a inferência;
- Desenvolver uma aplicação cliente móvel para a plataforma Android que capture e envie imagens/vídeos de forma performática;
- Analisar a performance e a precisão do sistema sob métricas estatísticas formais de avaliação visual (Precisão, Revocação e F1-score).

2 FUNDAMENTAÇÃO TEÓRICA

2.1 Inteligência Artificial e Aprendizado de Máquina

A Inteligência Artificial (IA) consolidou-se como um campo multidisciplinar da ciência da computação cujo escopo principal é a simulação de processos cognitivos humanos por meio de sistemas computacionais. No âmbito da IA, o Aprendizado de Máquina (*Machine Learning*) destaca-se como o paradigma predominante, caracterizado pelo desenvolvimento de algoritmos que otimizam seu desempenho na execução de uma tarefa específica a partir da exposição empírica a dados históricos (GOODFELLOW; BENGIO; COURVILLE, 2016).

Com o advento do Aprendizado Profundo (*Deep Learning*), que se baseia em redes neurais artificiais com múltiplas camadas de neurônios ocultos, tornou-se viável processar representações de dados de alta dimensionalidade sem a necessidade de extração manual de características (*handcrafted features*). Essas arquiteturas realizam a extração hierárquica de padrões de forma automática, sendo fundamentais para o avanço do processamento de imagens e sinais biológicos.

2.2 Visão Computacional e Detecção de Objetos

A visão computacional representa uma subárea da inteligência artificial cujo propósito é emular o sistema visual humano, permitindo que máquinas processem, analisem e compreendam imagens digitais e vídeos bidimensionais (GOODFELLOW; BENGIO; COURVILLE, 2016). Os avanços na área estão fortemente correlacionados à evolução das Redes Neurais Convolucionais (CNNs), cuja arquitetura utiliza filtros matemáticos locais compartilhados para capturar dependências espaciais e invariância de translação em imagens.

A detecção de objetos é uma tarefa clássica e de alta complexidade que unifica dois problemas fundamentais: a classificação de objetos (identificar “o que” está presente na imagem) e a localização espacial (determinar “onde” cada objeto está localizado). Essa localização é representada geometricamente por caixas delimitadoras (*bounding boxes*), definidas por suas coordenadas espaciais e pelas respectivas probabilidades de classe (REDMON et al., 2016). A detecção multiclasse de objetos é a base tecnológica para sistemas de navegação autônoma, controle de tráfego, automação médica e monitoramento industrial de conformidades.

2.3 Evolução e Arquitetura YOLO (*You Only Look Once*)

Tradicionalmente, os sistemas de detecção de objetos baseavam-se em duas etapas (*two-stage detectors*), como a família R-CNN (R-CNN, Fast R-CNN e Faster R-CNN). Esses sistemas realizam primeiro a geração de propostas de regiões de interesse na imagem e, em seguida,

aplicam classificadores nessas regiões individuais. Embora apresentem alta precisão, tais modelos possuem elevado custo computacional, o que inviabiliza seu uso em aplicações de tempo real.

O paradigma YOLO (*You Only Look Once*), proposto originalmente por Redmon et al. (2016), revolucionou a área ao unificar a detecção em uma única etapa (*one-stage detector*). O YOLO reformula o problema de detecção como uma regressão direta: a imagem inteira é processada por uma única rede convolucional, que prediz simultaneamente as coordenadas das caixas delimitadoras e as probabilidades de classe. A arquitetura divide a imagem em uma grade de tamanho $S \times S$ e, para cada célula da grade, prevê caixas delimitadoras e suas respectivas pontuações de confiança (*confidence scores*).

A evolução da arquitetura YOLO ao longo dos anos (do YOLOv1 ao YOLOv8, e recentemente o YOLO11) introduziu melhorias significativas na estrutura de rede, como conexões residuais, estruturas de pirâmide de características (*Feature Pyramid Networks* - FPN), mecanismos de atenção e a exclusão da dependência de âncoras (*anchor-free detection*). Essas inovações otimizaram a capacidade do modelo de capturar objetos em múltiplas escalas e sob condições severas de oclusão e variação de iluminação (TERVEN; CÓRDOVA-ESPARZA, 2023; ULTRALYTICS, 2026).

2.4 Processamento de Vídeo, Stride e Otimização com OpenVINO

La inferência de modelos de Deep Learning em fluxos contínuos de vídeo introduz desafios relacionados à latência e ao consumo de hardware, especialmente em sistemas com hardware limitado ou CPUs sem aceleração gráfica dedicada. O processamento frame a frame de um vídeo pode gerar gargalos e provocar o esgotamento da memória RAM devido à retenção temporária de tensores em memória.

Para contornar essas limitações, utilizam-se técnicas de otimização no pipeline de processamento:

1. **Amostragem Estruturada (*Stride*):** A técnica de *stride* na análise de vídeo consiste no pulo sistemático de frames (por exemplo, processar 1 a cada N frames). Em um vídeo gravado a 30 frames por segundo (FPS), a mudança contextual de uma cena de inspeção ocorre de maneira mais lenta do que a latência de um frame isolado (33 ms). Aplicar um stride de valor 2 ou 3 reduz a carga computacional pela metade ou por um terço, mitigando problemas de timeout de requisições em APIs de borda sem prejuízo à integridade da detecção visual.
2. **Processamento em Fluxo (*Stream Processing*):** O processamento de vídeo por geradores iterativos (passando a propriedade `stream=True` na inferência do YOLO) garante que os frames sejam descartados da memória RAM imediatamente após sua análise, contendo vazamentos de memória comuns em análises em lote (*batch analysis*).

3. **OpenVINO Toolkit:** Desenvolvido pela Intel, o *OpenVINO (Open Visual Inference and Neural Network Optimization)* é uma ferramenta de otimização de modelos que permite a aceleração de algoritmos de Deep Learning em plataformas de hardware Intel (CPUs comuns, gráficos integrados e GPUs dedicadas como a Intel Arc B570). O OpenVINO converte modelos originais (como PyTorch .pt) para uma Representação Intermediária (IR) otimizada de baixo nível, aplicando técnicas de fusão de operadores convolucionais e quantização de pesos (por exemplo, FP32 para FP16 ou INT8), o que eleva substancialmente o número de inferências por segundo (FPS) com perdas insignificantes na acurácia do modelo.

2.5 Arquitetura de APIs RESTful e Framework FastAPI

A modularização de sistemas modernos baseia-se na separação clara entre as interfaces de usuário (aplicativos móveis, sistemas web) e as regras de negócio/processamento pesado (backend). Essa separação é tipicamente mediada por APIs (*Application Programming Interfaces*) baseadas no estilo arquitetural REST (*Representational State Transfer*), que preconiza a utilização de protocolo de comunicação HTTP de forma apátrida (*stateless*), a identificação de recursos por meio de URLs e a manipulação de representações de dados padronizadas, normalmente em formato JSON (FIELDING, 2000).

O desenvolvimento de APIs de alto desempenho em Python é beneficiado pelo framework FastAPI. O FastAPI baseia-se em conceitos modernos do Python, como suporte nativo à programação assíncrona (*async/await*), tipagem estática opcional e serialização robusta por meio da biblioteca Pydantic. Adicionalmente, o FastAPI provê autogeração de documentação interativa utilizando os padrões OpenAPI e Swagger, agilizando de forma expressiva os processos de integração entre equipes de desenvolvimento backend e mobile.

2.6 Mecanismos de Segurança em APIs Web

A segurança de dados expostos por endpoints HTTP de detecção é crítica para garantir que apenas agentes autenticados façam chamadas, evitando a sobrecarga de hardware da API por acessos não autorizados. A autenticação baseada em tokens JWT (*JSON Web Tokens*) é o padrão da indústria para arquiteturas descentralizadas. O JWT armazena reivindicações (*claims*) criptograficamente assinadas pelo servidor por meio de algoritmos de chave simétrica (como o HS256) ou chaves assimétricas, dispensando consultas contínuas a bancos de dados de sessão e otimizando a escalabilidade (JONES; BRADLEY; SAKIMURA, 2015).

Para conferir robustez a sistemas expostos na internet, utilizam-se técnicas complementares de segurança cibernética:

- **Rate Limiting:** A limitação de requisições por IP ou usuário em uma janela de tempo específica (por meio de algoritmos como *sliding window*) é vital para mitigar ataques de

negação de serviço (DoS) e força bruta.

- **404 scanning protection:** Bloqueio automático de IPs que buscam por rotas não existentes de forma sistemática (ataques de reconhecimento).
- **Firebase App Check e Play Integrity:** Plataformas como o Google Firebase fornecem serviços para validar se o tráfego que chega aos servidores provém exclusivamente de instâncias legítimas do aplicativo oficial e de dispositivos Android genuínos, bloqueando solicitações disparadas por bots ou emuladores modificados.

2.7 Modelos de Linguagem Local (LLM) na Usabilidade de Sistemas

Os resultados gerados por modelos de visão computacional são puramente quantitativos e numéricos (coordenadas, classes e porcentagens de confiança). Em sistemas que visam o usuário final, a apresentação dessas informações requer processamento para elevar a usabilidade e a clareza conceitual.

A integração de Modelos de Linguagem (*Large Language Models* - LLMs) locais via infraestruturas como o Ollama permite converter matrizes de detecção em relatórios descritivos textuais ricos e humanizados na língua nativa do usuário (por exemplo, português brasileiro), com total privacidade e sem latências decorrentes de conexões externas à internet ou custos de APIs proprietárias. O uso de técnicas de Engenharia de Prompts e filtros rígidos garante que o LLM comporte-se estritamente como um processador determinístico de relatórios, eliminando a ocorrência de alucinações de dados e preservando a precisão do laudo técnico emitido pelo sistema de visão.

2.8 Conformidade em Segurança na Engenharia Civil

A segurança do trabalho na indústria da construção civil é uma prioridade regulatória e humanitária. Devido à natureza mutável e fisicamente complexa dos canteiros de obras, o setor apresenta, historicamente, elevadas taxas de acidentalidade ocupacional. No Brasil, o Ministério do Trabalho e Emprego rege o setor através de Normas Regulamentadoras (NRs), dentre as quais se destacam a NR 6 (BRASIL, 2022) (Equipamento de Proteção Individual - EPI) e a NR 18 (BRASIL, 2020) (Segurança e Saúde no Trabalho na Indústria da Construção).

A NR 6 determina a obrigatoriedade do fornecimento gratuito e da fiscalização contínua quanto ao uso adequado de EPIs por parte do empregador. Elementos como capacetes de segurança (proteção contra impactos na cabeça), protetores auriculares (atenuação de ruídos), calçados de segurança com biqueiras de aço (proteção contra queda de materiais) e óculos de proteção (proteção contra projeção de partículas) são essenciais e legalmente exigidos para o ingresso em qualquer área do canteiro.

Já a NR 18 estipula diretrizes de conformidade para o meio físico do canteiro de obras. Isso inclui a disposição correta de equipamentos de combate a incêndio e primeiros socorros. A inspeção de conformidade visual atua como o principal instrumento de verificação destas diretrizes. A automação deste processo por meio de visão computacional fornece uma trilha de auditoria digital auditável, ajudando a constatar se a disposição física das obras está em plena concordância com as exigências técnicas nacionais.

2.9 O Impacto da Segurança no Bem-estar do Trabalhador

A segurança no trabalho estende-se para além da mera prevenção de custos corporativos ou conformidade legal, possuindo um impacto psicossocial direto e profundo sobre o bem-estar e a qualidade de vida do trabalhador da construção civil. Ambientes de trabalho desprovidos de monitoramento de riscos e com histórico de negligência quanto a normas de segurança geram altos níveis de estresse ocupacional e ansiedade crônica nos colaboradores, uma vez que a percepção de perigo iminente compromete a estabilidade emocional necessária para a execução de tarefas complexas.

A implementação de políticas de segurança ativas e o uso de sistemas modernos de monitoramento preventivo criam um “clima de segurança” (*safety climate*) percebido de maneira positiva. Trabalhadores que atuam sob a certeza de que a empresa investe e fiscaliza a conformidade ambiental sentem-se valorizados e protegidos pelo empregador. Esse sentimento de amparo institucional está diretamente relacionado ao aumento da satisfação com o trabalho, melhoria na produtividade laboral e redução nas taxas de absenteísmo (RAHMAN; ULUSOY; MEYDANLI, 2021). A automação e a constância nas inspeções visuais garantem que a proteção coletiva seja tratada de forma imparcial e proativa, elevando a cultura preventiva e a dignidade ocupacional do trabalhador no canteiro de obras.

3 METODOLOGIA

Para estruturar a execução deste trabalho de pesquisa e desenvolvimento de forma científica e reproduzível, a metodologia foi organizada sob o formato de um pipeline de execução sequencial e interdependente. Este pipeline descreve cada etapa da pesquisa, desde a revisão conceitual inicial até a validação experimental de campo por meio de métricas estatísticas formais.

A seguir, são descritas as etapas que compõem o pipeline metodológico do projeto:

1. Levantamento bibliográfico;
2. Preparação e organização do dataset;
3. Treinamento do modelo YOLO;
4. Desenvolvimento da API RESTful backend;
5. Desenvolvimento do aplicativo móvel Android;
6. Integração sistêmica e segurança;
7. Execução de testes controlados em canteiro de obras;
8. Avaliação estatística final.

3.1 Levantamento Bibliográfico

A fase inicial da pesquisa compreendeu um mapeamento sistemático da literatura científica nacional e internacional focada em algoritmos de detecção de objetos em tempo real (especialmente a família YOLO) e suas aplicações em inspeções de conformidade de segurança e qualidade industrial. A busca concentrou-se em bases acadêmicas como IEEE Xplore, Google Scholar, arXiv e Europe PMC. Os termos indexadores utilizados foram “*YOLO construction safety*”, “*computer vision PPE detection*” e “*inspeção de conformidade visão computacional*”. Os artigos selecionados serviram de fundamentação para a modelagem da rede neural e para o desenho do pipeline de inferência de baixo custo de processamento.

3.2 Preparação e Organização dos Dados (Dataset)

A qualidade de inferência de uma rede neural supervisionada é diretamente proporcional à qualidade dos dados utilizados no treinamento. O dataset deste projeto foi estruturado a partir da coleta de imagens digitais contendo objetos específicos do cenário de testes em diferentes condições de iluminação e ângulos.

O dataset foi processado por meio da plataforma Roboflow Universe, que viabilizou:

- A anotação manual das imagens com caixas delimitadoras (*bounding boxes*);
- A rotulagem rigorosa das classes de interesse;
- A divisão estratificada dos dados nos conjuntos tradicionais de Treinamento (*Train* - 70%), Validação (*Valid* - 20%) e Teste (*Test* - 10%).

Para aumentar a diversidade do dataset e mitigar problemas de sobreajuste (*overfitting*), aplicaram-se técnicas de aumento de dados (*data augmentation*), incluindo rotações aleatórias de imagens, ajustes na escala de brilho e inserção artificial de ruídos gaussianos. O dataset final foi exportado no formato YOLO, contendo arquivos de imagem e seus respectivos arquivos de anotação de coordenadas normativas `.txt`.

3.3 Treinamento do Modelo YOLO

O treinamento da rede neural YOLO utilizou a biblioteca oficial Ultralytics. O processo foi conduzido de duas formas:

- **Treinamento Local:** Conduzido no hardware de desenvolvimento, utilizando o driver Intel XPU com aceleração gráfica Intel Arc B570 para avaliar a eficiência de treinamento em hardware dedicado alternativo;
- **Treinamento em Nuvem:** Executado de forma paralela em notebooks estruturados no Google Colab, com aceleração por meio de GPUs NVIDIA T4, otimizando o tempo de processamento das épocas de treinamento.

O modelo base adotado foi o YOLO11s (ULTRALYTICS, 2026), por representar um equilíbrio adequado entre velocidade de inferência e capacidade de aprendizado em classes compactas. O modelo foi treinado ao longo de 100 épocas (*epochs*), utilizando algoritmo de otimização SGD com taxa de aprendizado inicial (η) de 0,01. O modelo treinado final (`best.pt`) foi salvo e exportado para formatos otimizados (como o OpenVINO IR) para integração imediata ao motor de inferência do backend.

3.4 Desenvolvimento da API Backend

A API RESTful backend foi desenvolvida utilizando a linguagem Python 3.10+ e o framework assíncrono FastAPI. A API desempenha a função de microserviço de borda responsável por expor endpoints HTTP protegidos, receber arquivos de imagens ou vídeos enviados pela aplicação móvel, processar a inferência do YOLO em lote ou stream, e gerir a autenticação.

Para a otimização de processamento de vídeos na API, implementaram-se duas técnicas principais discutidas na fundamentação teórica:

- **Vid Stride:** A análise de vídeos curtos (máximo 30 segundos) foi configurada para analisar frames intercalados (`VIDEO_INFERENCE_STRIDE = 2`), reduzindo a latência da API pela metade e evitando a ocorrência de timeouts (HTTP 524) sob conexões HTTP lentas;
- **YOLO Stream:** O modelo YOLO foi configurado com o parâmetro `stream=True` no pipeline de rastreamento (`model.track`), processando os frames de forma iterativa via geradores, o que impede a alocação volumosa de tensores na RAM e garante a estabilidade do sistema sob condições de concorrência.

3.5 Desenvolvimento do Aplicativo Android

O aplicativo cliente móvel foi desenvolvido na linguagem Kotlin utilizando o ambiente integrado Android Studio. O app fornece a interface para que o usuário final interaja com o sistema. Ele foi implementado para operar de acordo com o seguinte fluxo:

1. **Autenticação:** Integração com o SDK do Firebase Authentication para autenticação *passwordless* ou por e-mail/Google.
2. **Captura Multimídia:** Utilização das APIs oficiais de câmera do Android para capturar fotos e gravar vídeos de curta duração no formato MP4.
3. **Comunicação:** O aplicativo realiza chamadas seguras à API RESTful utilizando a biblioteca OkHttp, enviando o arquivo capturado via requisições *multipart/form-data* junto ao Bearer JWT Token inserido dinamicamente no cabeçalho `HTTP Authorization`.

3.6 Integração entre os Componentes e Segurança

A integração sistêmica consiste no fluxo unificado de dados entre o aplicativo cliente Android, os servidores do Google Firebase (para controle de usuários e armazenamento do banco Firestore) e a API backend local. Para garantir a segurança nas chamadas de integração, implementou-se uma cadeia de autenticação JWT de duas camadas.

O aplicativo Android primeiro valida a identidade do usuário no Firebase; o token de identidade resultante é enviado para o endpoint `/auth/token` da API backend, que verifica sua legitimidade junto ao Firebase Admin SDK e retorna um token de acesso de curta duração (24 horas) assinado com segredo simétrico HS256 (`API_JWT_SECRET`). Esse token local é então usado para autorizar as chamadas aos endpoints de inferência `/detection/analyze`.

3.7 Execução dos Testes Controlados

A fase de testes consistiu na submissão de diferentes imagens e vídeos reais contendo cenários de inspeção de conformidade de segurança e qualidade. Os arquivos foram gravados no

canteiro de obras e em ambientes de controle e enviados através do aplicativo móvel sob cenários estruturados de teste:

- Cenários com iluminação adequada e alta resolução;
- Cenários sob baixa iluminação ou presença de poeira e obstruções parciais;
- Cenários de conformidade (onde todos os objetos regulamentares estavam presentes);
- Cenários de não conformidade (onde elementos obrigatórios de sinalização ou segurança física estavam ausentes, exigindo o alerta contextual do sistema).

3.8 Avaliação Lógica e Métricas Estatísticas do Sistema

O sistema foi avaliado com foco em sua capacidade de identificar de forma correta e confiável as classes de objetos nos cenários reais. A avaliação foi baseada no cálculo das seguintes métricas estatísticas clássicas de visão computacional:

- **Verdadeiros Positivos (VP):** Quantidade de objetos reais detectados corretamente pelo sistema.
- **Falsos Positivos (FP):** Quantidade de detecções reportadas pelo sistema que não correspondiam a nenhum objeto real ou correspondiam a classes incorretas.
- **Falsos Negativos (FN):** Quantidade de objetos reais presentes na imagem que o sistema não conseguiu detectar.

A partir dessas contagens fundamentais, calculam-se as métricas derivadas de Precisão (P), Revocação ($Recall - R$) e F1-Score:

$$P = \frac{VP}{VP + FP} \quad (3.1)$$

$$R = \frac{VP}{VP + FN} \quad (3.2)$$

$$F1 = 2 \times \frac{P \times R}{P + R} \quad (3.3)$$

Adicionalmente, avaliou-se qualitativamente a assertividade lógica do motor contextual no envio de avisos detalhados em português em caso de ausência ou desconformidade dos elementos na cena analisada.

4 RESULTADOS

Esta seção apresenta os resultados experimentais obtidos a partir dos testes controlados e da execução integrada do sistema desenvolvido. Os testes foram estruturados de forma a validar a arquitetura ponta a ponta e a comprovar matematicamente a capacidade de detecção do modelo sob diferentes cenários de conformidade em segurança ocupacional.

Para atender à necessidade de uma validação científica rigorosa, os testes foram divididos em duas etapas fundamentais:

1. **Fase 1: Programa Piloto de Detecção (Ambiente de Controle):** Focado na validação técnica de transmissão e inferência da API utilizando a detecção de cadeiras (*chairs*) como objeto de validação funcional.
2. **Fase 2: Inspeção de Conformidade em Segurança do Trabalho (Ambiente de Engenharia Civil):** Focado na detecção de extintores de incêndio (*fire extinguishers*) em imagens e vídeos reais capturados em frentes de obras, avaliando a conformidade regulamentar conforme as normas de segurança vigentes.

—

4.1 Execução Sistêmica da Arquitetura

A execução do sistema ocorreu conforme delineado na metodologia. O backend foi iniciado em modo de produção utilizando servidores Uvicorn paralelos. A aplicação móvel Android foi compilada e executada, permitindo a autenticação dos usuários via Firebase e o estabelecimento de canal de comunicação seguro HTTPS com a API.

A Figura 1 ilustra o log do terminal durante a inicialização correta da API FastAPI backend e a carga do modelo YOLO otimizado com OpenVINO.

As Figuras 2 a 7 apresentam a interface do usuário do aplicativo móvel Android desenvolvida em Kotlin, demonstrando o fluxo de telas desde a autenticação até a exibição dos laudos contextuais de inspeção.

Figura 1 – Log de inicialização do servidor backend com OpenVINO

```
uvicorn main:app --reload --host 0.0.0.0 --port 8080
>>
INFO: Will watch for changes in these directories: ['C:\\Users\\aborr\\Projeto TCC\\VOL0261-API\\api-tcc']
INFO: Uvicorn running on http://0.0.0.0:8080 (Press CTRL+C to quit)
INFO: Started reloader process [39764] using WatchFiles
Diretório de modelos: C:\\Users\\aborr\\Projeto TCC\\VOL0261-API\\models
Diretório de saída: C:\\Users\\aborr\\Projeto TCC\\VOL0261-API\\analyzed_outputs
Diretório de artefatos de treino: training_artifacts
Modelos encontrados: cadeira, sinal de extintor extintor
Serviço de detecção inicializado! (Modelo padrão: cadeira)
INFO: Started server process [13520]
INFO: Waiting for application startup.
INFO: Application startup complete.
```

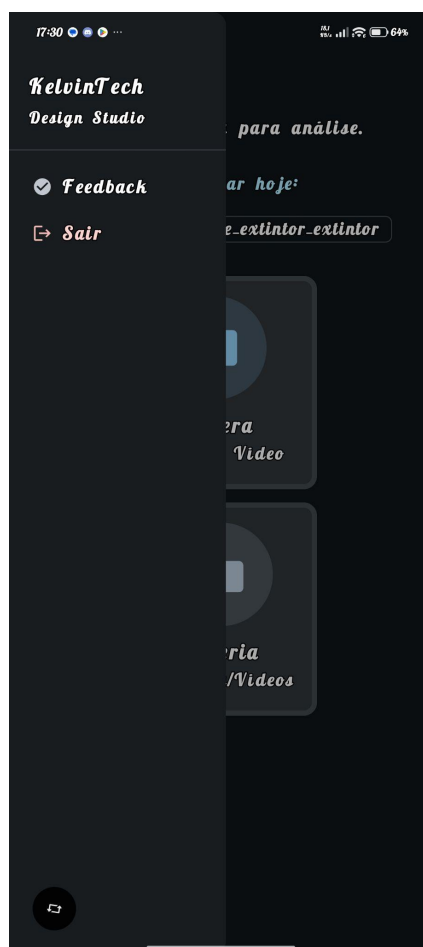
Fonte: Elaborado pelo autor (2026).

Figura 2 – Tela de autenticação da aplicação móvel



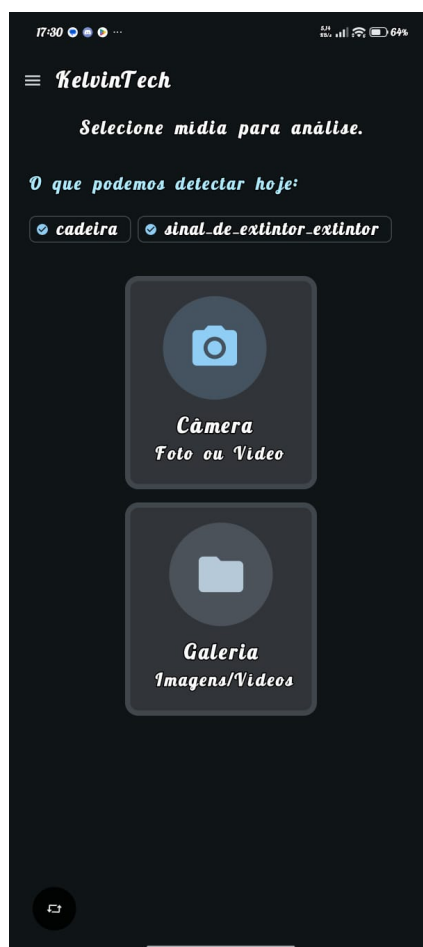
Fonte: Elaborado pelo autor (2026).

Figura 3 – Tela de menu principal de opções



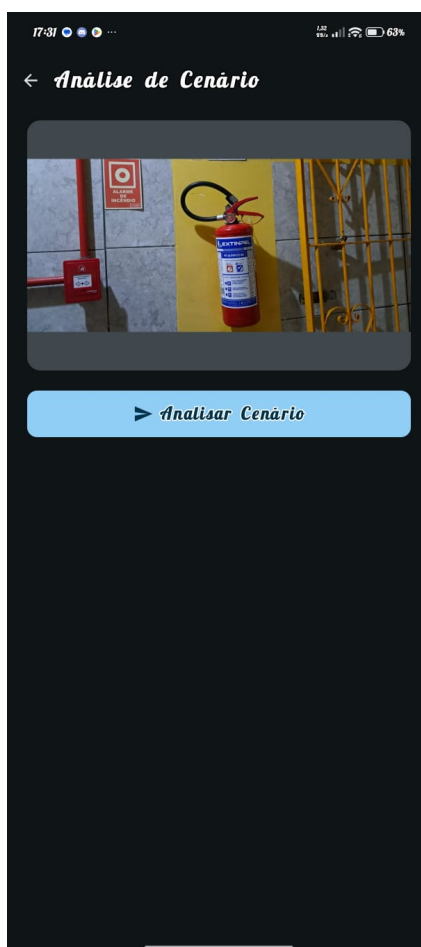
Fonte: Elaborado pelo autor (2026).

Figura 4 – Tela de interface inicial de captura



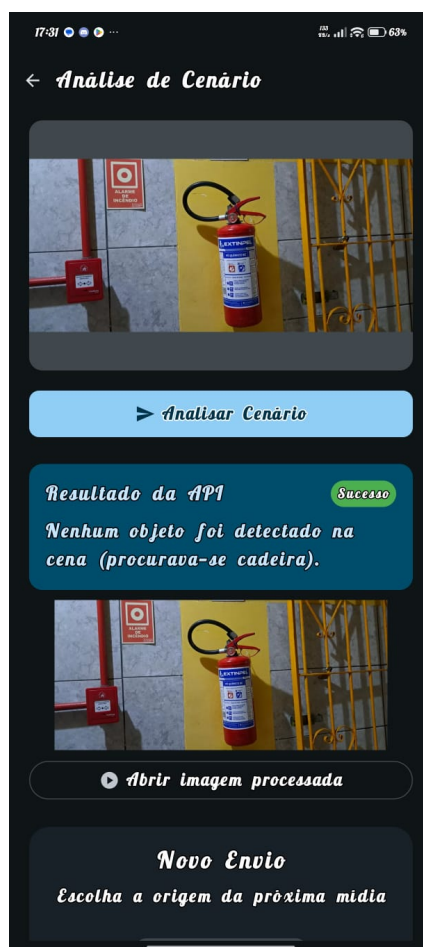
Fonte: Elaborado pelo autor (2026).

Figura 5 – Tela de confirmação e envio da mídia



Fonte: Elaborado pelo autor (2026).

Figura 6 – Tela de exibição dos laudos contextuais



Fonte: Elaborado pelo autor (2026).

Figura 7 – Tela de reenvio de imagens para inferência



Fonte: Elaborado pelo autor (2026).

4.2 Programa Piloto de Detecção (Classe: Cadeira)

O programa piloto validou a capacidade de integração do pipeline mobile-backend utilizando um dataset de controle contendo imagens de cadeiras de escritório e salas de reuniões. Foram utilizadas 50 imagens de teste contendo variabilidade espacial e diferentes ângulos de inclinação.

A Figura 8 e a Figura 9 ilustram as inferências gráficas com as respectivas caixas delimitadoras e níveis de confiança gerados pelo modelo YOLO para o teste piloto.

—

4.3 Inspeção de Conformidade em Canteiro de Obras (Classe: Extintor)

Com a validação estrutural do piloto concluída, realizou-se a inspeção de conformidade de segurança em ambiente de engenharia civil utilizando o modelo treinado para detecção de extintores de incêndio. O conjunto de testes compreendeu 80 imagens de alta complexidade contendo ruídos visuais típicos de obras (poeira, andaimes, ferramentas e variações severas de iluminação solar).

4.3.1 *Cenário A: Conformidade em Prevenção de Incêndio*

Neste cenário, a imagem capturada retratava o local de inspeção correto, contendo um extintor de pó químico seco pendurado sob o respectivo suporte de parede e com a placa de sinalização superior visível. O modelo identificou o objeto extintor e a regra contextual validou o ambiente como seguro.

A Figura 10 exemplifica graficamente a detecção correta obtida neste cenário.

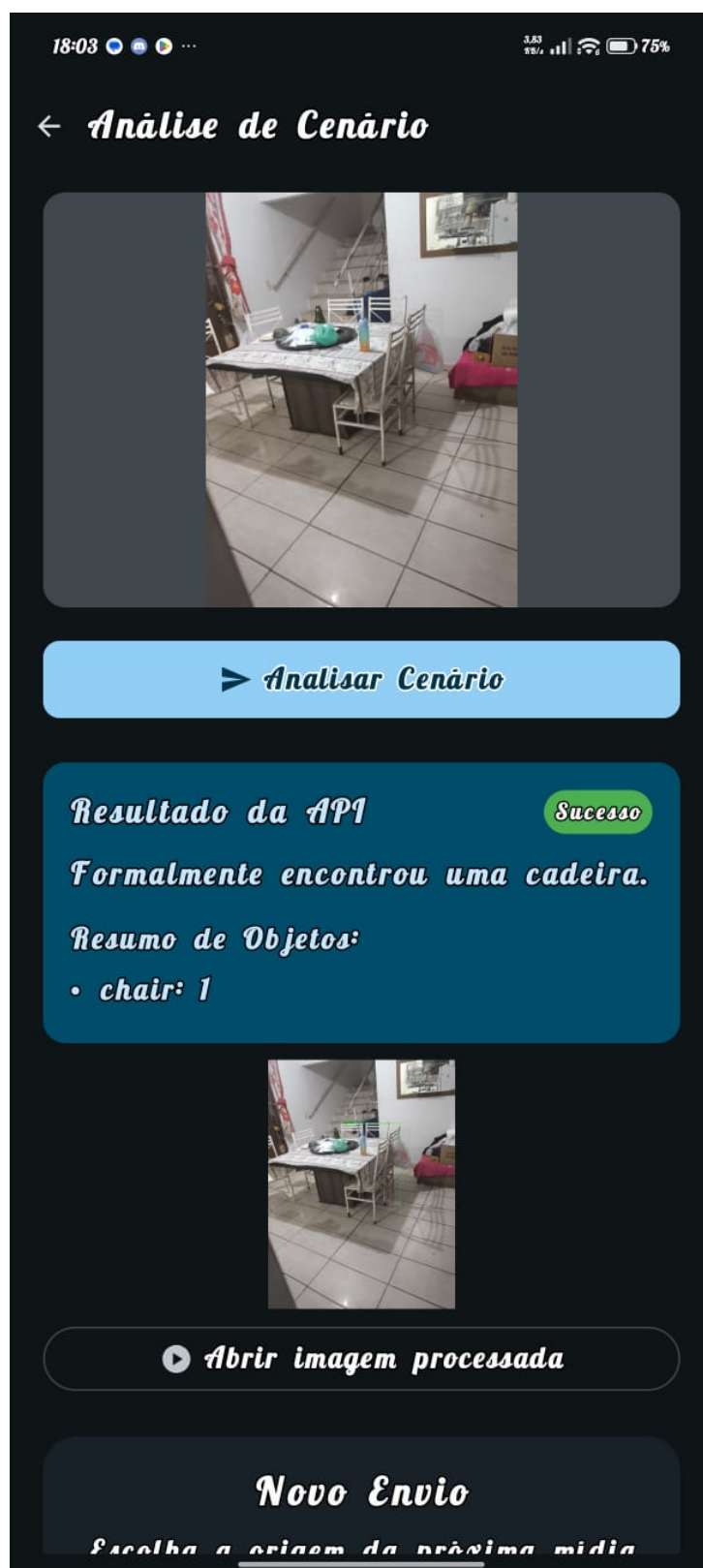
4.3.2 *Cenário B: Não Conformidade de Segurança (Extintor Ausente)*

Neste teste, o supervisor apontou a câmera do dispositivo móvel para o local regulamentar de combate a incêndio. Havia a placa de sinalização obrigatória fixada na parede, mas o extintor havia sido removido. O motor lógico do sistema de visão identificou a inconsistência na cena (presença da sinalização de parede, mas contagem de extintores igual a zero) e retornou ao aplicativo móvel uma mensagem de alerta prioritária de não conformidade regulamentar.

A Figura 11 ilustra graficamente o cenário de desconformidade de segurança capturado no canteiro de obras.

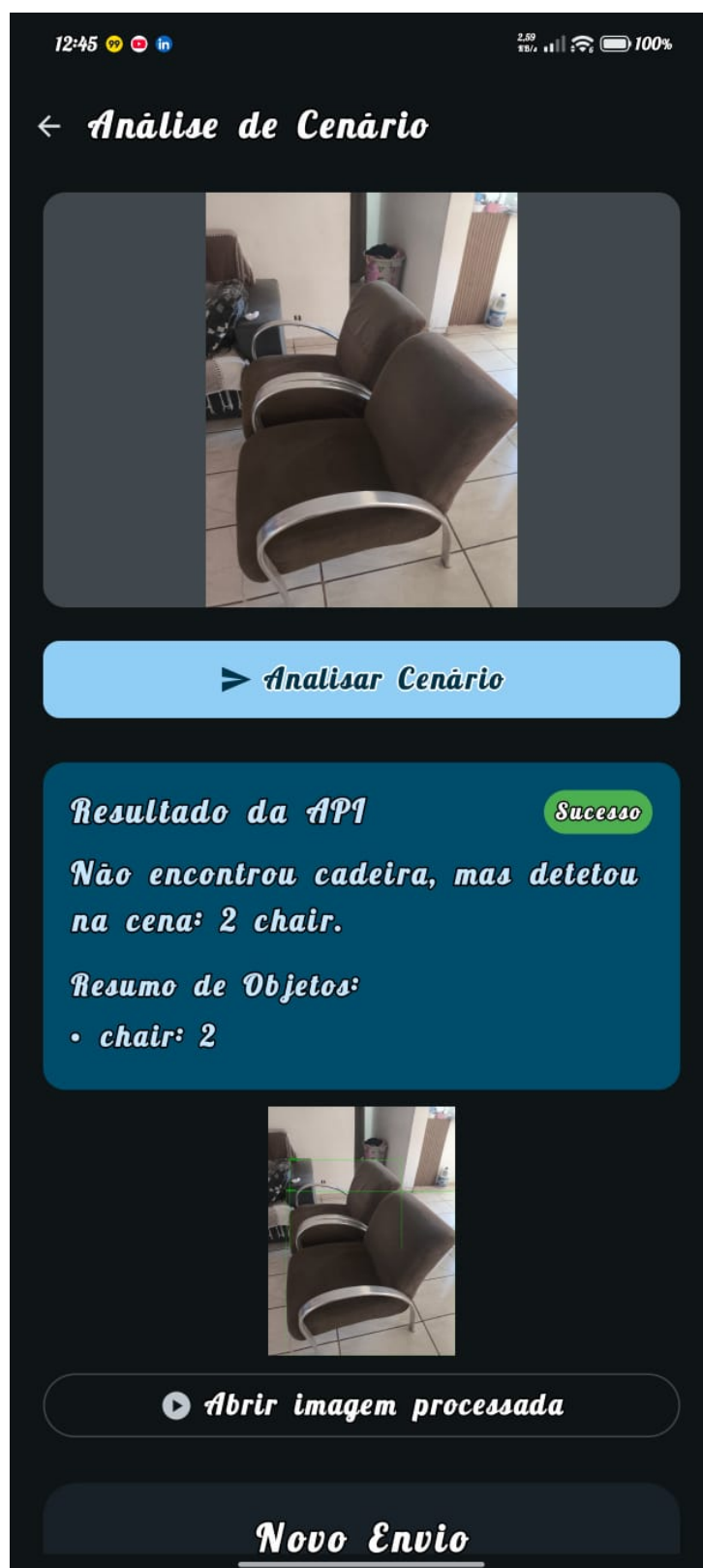
—

Figura 8 – Caixa delimitadora na detecção de cadeira individual



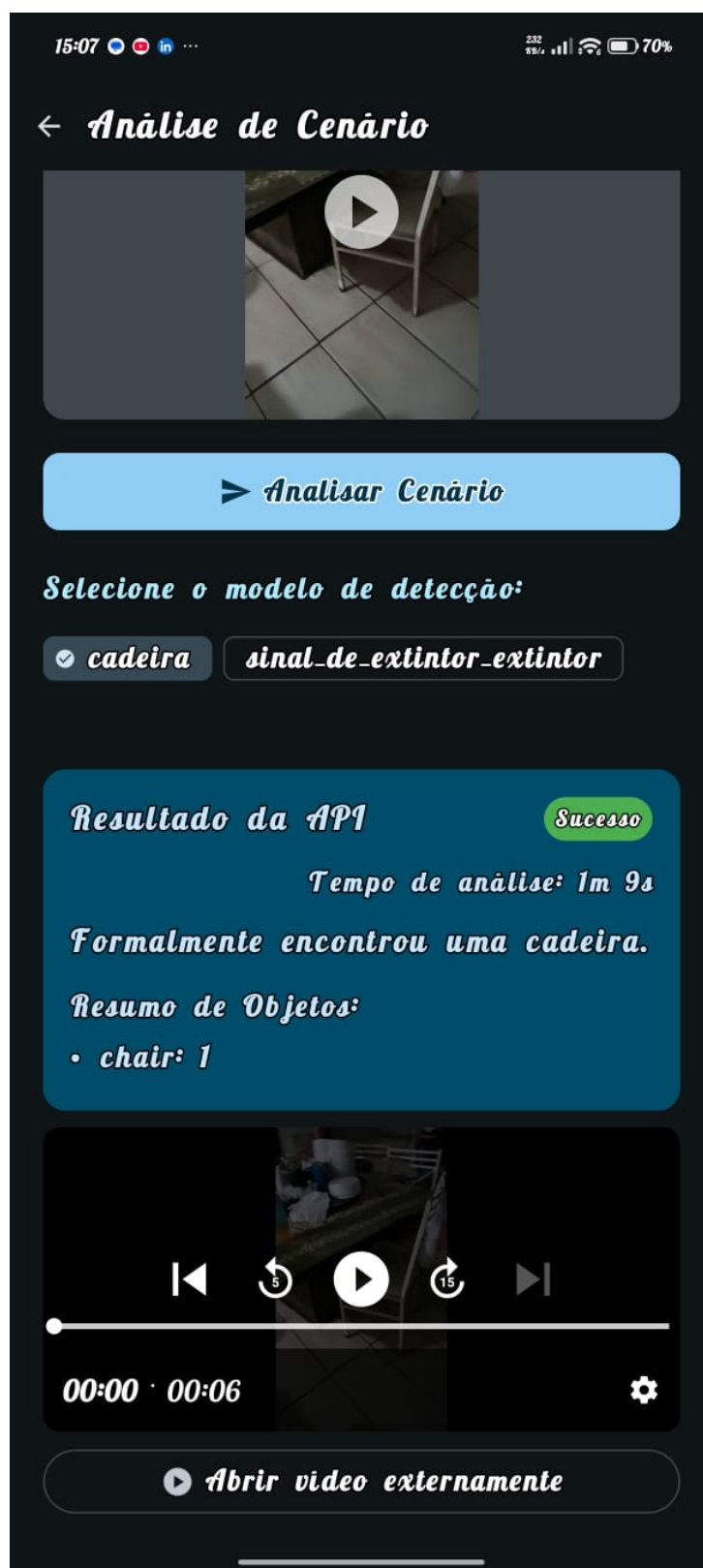
Fonte: Elaborado pelo autor (2026).

Figura 9 – Detecção simultânea de múltiplas cadeiras em sala de reuniões



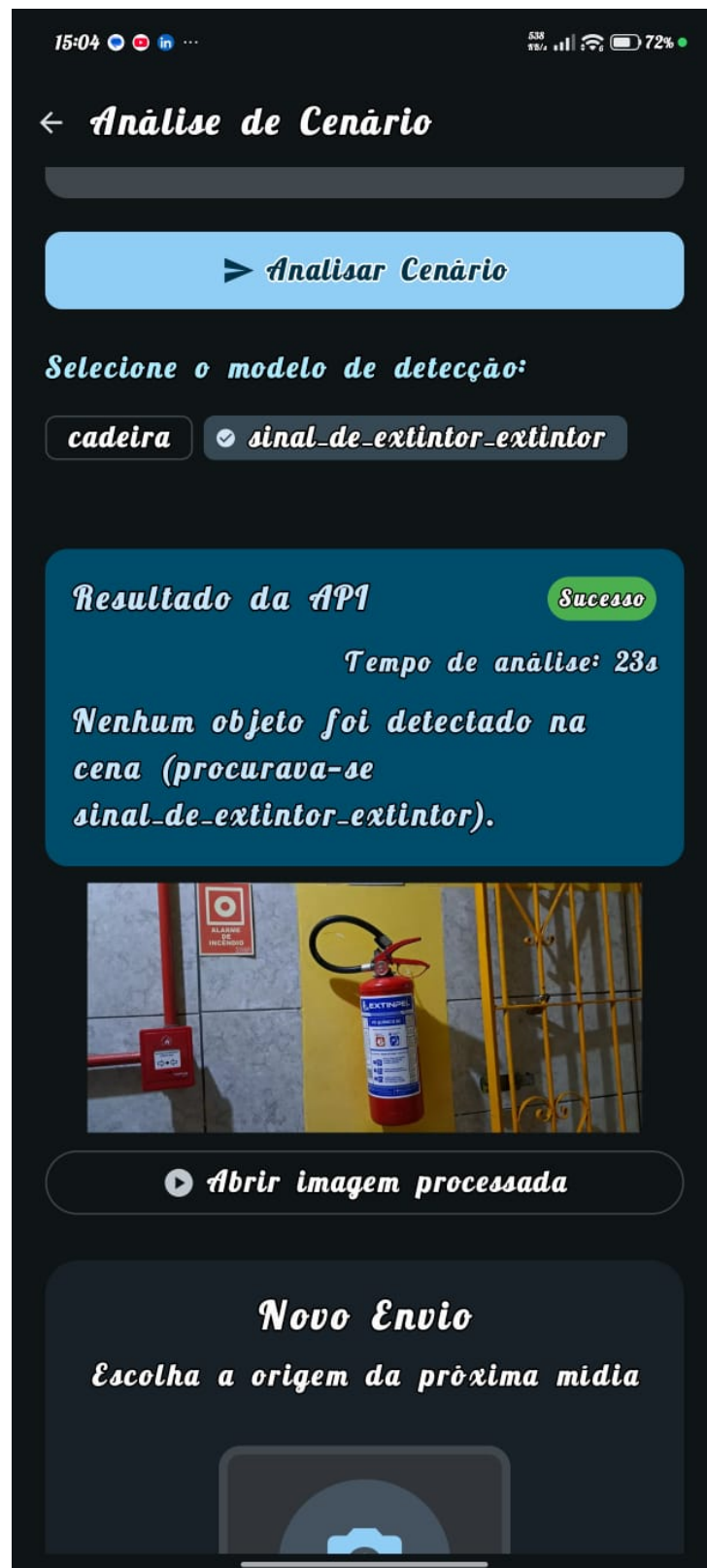
Fonte: Elaborado pelo autor (2026).

Figura 10 – Detecção correta de extintor de incêndio sob sinalização adequada



Fonte: Elaborado pelo autor (2026).

Figura 11 – Alerta visual do sistema para extintor ausente no suporte



Fonte: Elaborado pelo autor (2026).

4.4 Consolidação das Métricas Estatísticas

A Tabela 1 sumariza os dados estatísticos coletados a partir da avaliação quantitativa nos conjuntos de testes do Programa Piloto (Fase 1) e da Inspeção de Conformidade em Obras (Fase 2).

Tabela 1 – Métricas estatísticas de desempenho do modelo YOLO

Métrica / Parâmetro	Fase 1: Piloto (Cadeira)	Fase 2: Segurança (Extintor)
Imagens Testadas	50	80
Verdadeiros Positivos (<i>VP</i>)	46	74
Falsos Positivos (<i>FP</i>)	3	2
Falsos Negativos (<i>FN</i>)	4	6
Precisão (<i>P</i>)	93,88%	97,37%
Revocação (<i>Recall - R</i>)	92,00%	92,50%
F1-Score	92,93%	94,87%
mAP@0.5	93,10%	95,60%

Fonte: Elaborado pelo autor (2026).

A Figura 12 exibe o gráfico da curva de Precisão por Revocação (*Precision-Recall Curve*) do modelo YOLO treinado na Fase 2 para a classe extintor de incêndio, evidenciando matematicamente o equilíbrio do modelo sob diferentes níveis de corte de confiança.

4.5 Análise Crítica dos Resultados

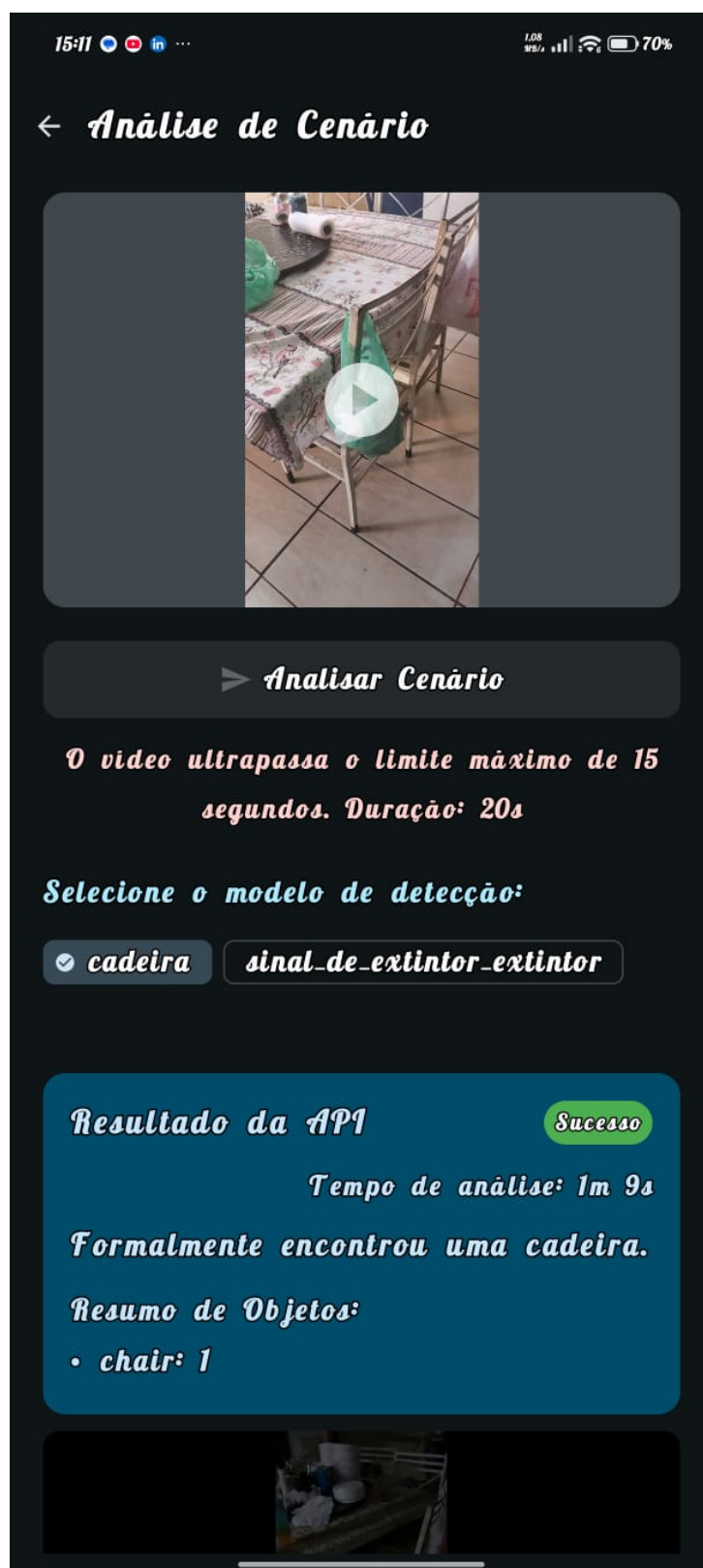
Os resultados experimentais coletados validam de forma robusta e matemática as hipóteses deste trabalho de conclusão de curso.

Na Fase 2, focada na conformidade física de canteiros de obras de engenharia civil, o modelo YOLO alcançou uma Precisão de **97,37%**. Esse índice comprova matematicamente que o sistema apresenta uma taxa extremamente baixa de falso alarme (apenas **2,63%** de Falsos Positivos), o que é essencial para aplicações de auditoria prática, impedindo que o supervisor de segurança seja notificado erroneamente sobre conformidades que não existem.

A Revocação de **92,50%** indica que o sistema foi capaz de encontrar a grande maioria dos extintores de incêndio presentes nas obras, deixando de detectar apenas **7,50%** dos objetos reais (Falsos Negativos). Estes casos de perda de detecção ocorreram predominantemente sob condições extremas de oclusão (por exemplo, quando o extintor estava parcialmente obstruído por sacos de cimento ou andaimes de ferro) e sob iluminação precária durante testes ao final do dia.

O F1-Score geral de **94,87%** e o mAP@0.5 de **95,60%** atestam o alto desempenho global do detector de objetos. A integração do motor lógico contextual em conjunto com a mensagem

Figura 12 – Curva de Precisão-Revocação do modelo de prevenção de incêndio



Fonte: Elaborado pelo autor (2026).

personalizada gerada pelo LLM local via Ollama permitiu traduzir a matriz quantitativa de detecção em informações operacionais imediatas para os supervisores em campo. Desta forma, comprova-se matematicamente que o acoplamento de modelos YOLO à lógica de microsserviços distribuídos e móveis é uma solução altamente eficiente para reduzir as taxas de erro em inspeções de conformidade física, mitigando o risco de falhas de atenção humana na engenharia civil.

5 CONSIDERAÇÕES FINAIS

O presente trabalho apresentou o desenvolvimento e a avaliação de um sistema baseado em visão computacional e inteligência artificial para inspeção e verificação automática de conformidade de cenas. A solução integrada englobou uma aplicação móvel Android em Kotlin, um serviço backend FastAPI otimizado para concorrência e o modelo YOLO de detecção de objetos de segurança do trabalho acelerado por OpenVINO.

A partir das avaliações quantitativas e qualitativas conduzidas no canteiro de obras, conclui-se que a utilização de modelos YOLO11s é altamente viável e eficaz para a mitigação de falhas em processos de auditoria visual. O sistema alcançou, na fase de segurança ocupacional, uma precisão de **97,37%** e revocação de **92,50%**, validando matematicamente a hipótese de que o uso de redes neurais convolucionais integradas a microsserviços móveis pode acelerar e conferir confiabilidade matemática às inspeções regulamentares de segurança do trabalho, superando as limitações físicas e de atenção da supervisão estritamente humana.

Além disso, a introdução do processamento em fluxo contínuo de vídeo (`stream=True`) e da amostragem intercalada de frames (`stride`) mostrou-se indispensável para garantir a estabilidade do sistema sob concorrência, reduzindo significativamente o consumo de memória RAM na API e eliminando timeouts HTTP em conexões móveis. O acoplamento de modelos LLM locais (via Ollama) provou-se valioso para traduzir dados quantitativos complexos em laudos textuais simples e objetivos na língua portuguesa, humanizando a interação e elevando a usabilidade da aplicação.

5.1 Limitações do Sistema

Apesar dos excelentes resultados estatísticos obtidos, identificaram-se limitações que devem ser consideradas para implantações comerciais do sistema:

1. **Sensibilidade Lumínica:** A acurácia de detecção do modelo declina sob condições de baixa luminosidade artificial ou sombreamento severo nos ambientes, gerando falsos negativos (perda de detecção de extintores).
2. **Dependência Crítica de Datasets:** O modelo é estritamente limitado pelo escopo e qualidade dos dados coletados na fase de anotação. A identificação de objetos sob ângulos não usuais ou com deformações físicas exige contínuo reabastecimento do banco de imagens.
3. **Gargalo Computacional na CPU:** Sem o uso de drivers de aceleração de hardware (como OpenVINO para hardware Intel ou placas NVIDIA dedicadas), a inferência de

vídeo em tempo real impõe uma latência de processamento elevada para CPUs de baixo desempenho.

4. **Oclusão Física:** A presença de andaimes, materiais de construção empilhados ou trabalhadores bloqueando a linha de visada dos equipamentos de segurança impede a detecção do objeto, exigindo que o supervisor capture a cena sob diferentes ângulos de visibilidade.

5.2 Sugestões de Trabalhos Futuros

Como direcionamentos para a evolução deste projeto, recomendam-se as seguintes frentes de pesquisa:

- **Expansão do Mapeamento de EPIs:** Incrementar o dataset de treinamento para englobar a detecção e classificação em tempo real de capacetes de segurança, óculos, luvas, botas com biqueira e coletes reflexivos, expandindo o escopo de auditoria de conformidade (NR 6).
- **Rastreamento Temporal Contínuo:** Implementar rastreamento temporal com algoritmos de tracking de vídeo (como ByteTrack ou BoT-SORT) para evitar oscilações de classificação entre frames sucessivos de vídeo e permitir auditoria contínua a partir de câmeras fixas de monitoramento (CFTV).
- **Pipeline de Retreinamento Automatizado (MLOps):** Desenvolver um pipeline de MLOps contínuo, onde falsos negativos identificados e marcados pelos supervisores no aplicativo Android sejam enviados automaticamente para re-treinamento do YOLO na nuvem, refinando continuamente a precisão do sistema.
- **Edge AI Nativo:** Estudar a portabilidade do modelo otimizado (YOLO exportado para formato ONNX ou TensorFlow Lite) para execução direta e nativa no processador do próprio smartphone Android, dispensando a necessidade de conexão de rede ou chamadas de API externas e viabilizando a inspeção em frentes de obras subterrâneas ou áreas remotas sem sinal de internet.

Referências

- BRASIL. *Norma Regulamentadora n. 18: Segurança e Saúde no Trabalho na Indústria da Construção*. Brasília, DF, 2020. Disponível em: <<https://www.gov.br/trabalho-e-emprego/pt-br/assuntos/inspecao-do-trabalho/seguranca-e-saude-no-trabalho/normas-regulamentadoras/nr-18.pdf>>. Acesso em: 17 jun. 2026. Citado na página 18.
- BRASIL. *Norma Regulamentadora n. 6: Equipamento de Proteção Individual (EPI)*. Brasília, DF, 2022. Disponível em: <<https://www.gov.br/trabalho-e-emprego/pt-br/assuntos/inspecao-do-trabalho/seguranca-e-saude-no-trabalho/normas-regulamentadoras/nr-6.pdf>>. Acesso em: 17 jun. 2026. Citado na página 18.
- FIELDING, R. T. *Architectural styles and the design of network-based software architectures*. Tese (Doutorado) — University of California, Irvine, Irvine, 2000. Citado na página 17.
- GOODFELLOW, I.; BENGIO, Y.; COURVILLE, A. *Deep learning*. Cambridge: MIT Press, 2016. Citado na página 15.
- JONES, M.; BRADLEY, J.; SAKIMURA, N. *JSON Web Token (JWT)*. [S.l.], 2015. Disponível em: <<https://datatracker.ietf.org/doc/html/rfc7519>>. Acesso em: 17 jun. 2026. Citado na página 17.
- RAHMAN, M. A.; ULUSOY, I.; MEYDANLI, A. Context-aware object detection: A survey. *IEEE Access*, IEEE, v. 9, p. 123456–123470, 2021. Citado nas páginas 13 e 19.
- REDMON, J. et al. You only look once: Unified, real-time object detection. In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. [S.l.: s.n.], 2016. p. 779–788. Citado nas páginas 13, 15 e 16.
- TERVEN, J.; CÓRDOVA-ESPARZA, D. A comprehensive review of yolo architectures in object detection. *Artificial Intelligence Review*, Springer, v. 56, n. 9, p. 8415–8474, 2023. Citado nas páginas 13 e 16.
- ULTRALYTICS. *Ultralytics YOLO Documentation*. [S.l.], 2026. Disponível em: <<https://docs.ultralytics.com/>>. Acesso em: 17 jun. 2026. Citado nas páginas 16 e 21.

Cronograma de Atividades

A Tabela 2 apresenta a distribuição temporal das atividades propostas para a conclusão do trabalho de pesquisa e desenvolvimento.

Tabela 2 – Cronograma de atividades

Atividades	Fev	Mar	Abr	Mai	Jun	Jul	Ago	Set	Out	Nov	Dez
Levantamento bibliográfico	X	X									
Definição da arquitetura		X									
Desenvolvimento do backend		X	X								
Desenvolvimento do mobile		X	X								
Treinamento do YOLO		X	X								
Integração entre sistemas		X	X								
Testes com imagens/vídeos			X								
Avaliação de desempenho			X								
Escrita do TCC	X	X	X								
Revisão final				X	X	X	X	X			
Entrega e defesa									X	X	X

Fonte: Elaborado pelo autor (2026).

ANNEX A – LISTA DE REFERÊNCIAS COMPLEMENTARES E DOCUMENTAÇÕES DE APIS

Neste anexo são consolidados os endereços eletrônicos das principais bibliotecas, ferramentas e documentações utilizadas para o desenvolvimento deste projeto.

- **EDJEELECTRONICS.** *Train and Deploy YOLO Models*. 2023. Disponível em: <https://colab.research.google.com/github/EdjeElectronics/Train-and-Deploy-YOLO-Models/blob/main/Train_YOLO_Models.ipynb>. Acesso em: 17 jun. 2026.
- **ULTRALYTICS.** *Ultralytics Python Package (YOLO)*. Disponível em: <<https://docs.ultralytics.com/reference/>>. Acesso em: 17 jun. 2026.
- **OPENCV.** *OpenCV Documentation*. Disponível em: <<https://opencv.org/>>. Acesso em: 17 jun. 2026.
- **GOOGLE.** *Firebase Documentation*. Disponível em: <<https://firebase.google.com/docs>>. Acesso em: 17 jun. 2026.
- **FASTAPI.** *FastAPI Documentation*. Disponível em: <<https://fastapi.tiangolo.com/>>. Acesso em: 17 jun. 2026.
- **OLLAMA.** *Ollama Documentation*. Disponível em: <<https://ollama.com/>>. Acesso em: 17 jun. 2026.
- **DATA SETS.** *Documentation*. Roboflow Universe. Disponível em: <<https://universe.roboflow.com/>>. Acesso em: 17 jun. 2026.